RFC-001: Cloud-Native Application Delivery Platform

Purpose

This RFC proposes a cloud-native architecture for hosting and operating a highly available microservice platform on Microsoft Azure in response to the supplied design exercise. The objective is to demonstrate how modern DevOps, Infrastructure as Code (IaC), Continuous Integration (CI), GitOps and Kubernetes practices can be combined to deliver a secure, scalable and maintainable application platform.

The proposed solution addresses the requirements of three independent workloads:

Auth Service: A highly available Java-based authentication and authorisation microservice delivered through a CI pipeline as a Java JAR. The service validates inbound HTTP requests against a PostgreSQL database before forwarding authorised requests to downstream services.

Main Application: A highly available Node.js application delivered through a CI pipeline as a packaged application. The service processes periodic requests from mobile clients while performing read and write operations against a Cassandra database.

Cleanup Job: A scheduled Python application delivered through the CI pipeline as a Python script. The workload executes periodically to archive historical data within the same Cassandra database used by the Main Application, ensuring long-term data maintenance without impacting the primary application workload.

This proposal extends beyond the minimum implementation by demonstrating how these workloads integrate into an enterprise Azure platform using containerisation, Kubernetes orchestration, GitHub Actions, Terraform, GitOps deployment with ArgoCD, and a comprehensive observability stack consisting of Prometheus, Grafana, Loki and Alertmanager.

Several architectural assumptions have been made regarding shared enterprise services. These include Azure Kubernetes Service (AKS), networking, identity, monitoring and databases. These assumptions are explicitly documented and are intended to illustrate how the implemented solution would integrate into a production-ready Azure environment rather than to imply that these shared platform components are provisioned within this repository.

The proposed architecture has been designed with these core primary considerations in mind:

Security: through Infrastructure as Code, OpenID Connect (OIDC) authentication, Azure RBAC, container image scanning, GitOps deployment and least-privilege access.

Scalability: through containerised microservices, Kubernetes orchestration, health probes, replica-based deployments and support for Horizontal Pod Autoscaling.

Maintainability: through reusable Terraform modules, modular GitHub Actions workflows, Python operational automation, GitOps principles and a well-structured repository layout.

Cost Effectiveness: by provisioning only the resources required by the exercise, leveraging existing enterprise platform services where appropriate, and adopting open-source technologies such as Terraform, ArgoCD, Prometheus, Grafana, Loki, Alertmanager, Trivy and Renovate to minimise licensing costs while maintaining enterprise-grade capabilities.

The purpose of this RFC is to document the architectural decisions, implementation approach, assumptions and technology integrations that together form a secure, automated and operationally resilient cloud-native application delivery platform.

Context

The platform consists of three independent workloads:

- **Auth Service** – A Java microservice responsible for authenticating and authorising inbound requests using PostgreSQL.
- **Main Application** – A Node.js application responsible for servicing mobile application requests and interacting with Cassandra.
- **Cleanup Job** – A scheduled Python application responsible for archiving historical data within Cassandra.

The repository contains the application source code, Terraform modules, GitHub Actions workflows, Kubernetes manifests, ArgoCD configuration, monitoring configuration and operational automation scripts.

Infrastructure provisioning is intentionally limited to Azure resources required by the exercise, while enterprise platform components are assumed to already exist.

Assumptions

To keep the exercise focused on application delivery rather than enterprise platform provisioning, the following services are assumed to already exist:

- Azure Subscription
- Microsoft Entra ID
- OpenID Connect (OIDC) Federation
- Azure Kubernetes Service (AKS)
- Virtual Network
- Azure Firewall
- Network Security Groups
- Azure Load Balancer
- Ingress Controller
- Azure DNS
- PostgreSQL
- Cassandra
- Prometheus
- Grafana
- Loki
- Alertmanager
- Slack Workspace

These shared services would typically be provided through an enterprise Azure Landing Zone.

Architecture

The solution is divided into four logical layers:

1. **Infrastructure** – Azure resources provisioned using Terraform.
2. **Continuous Integration** – GitHub Actions builds, validates and publishes application artefacts.

3. **GitOps Deployment** – ArgoCD continuously synchronises Kubernetes manifests from Git into AKS.
4. **Operations** – Prometheus, Grafana, Loki and Alertmanager provide monitoring, logging and alerting.

Each layer has a clearly defined responsibility while remaining fully integrated through automation.

Infrastructure

Infrastructure provisioning is managed using reusable Terraform modules.

Current modules provision:

- Azure Resource Group
- Azure Container Registry

Example:

```
module "resource_group" {
  source   = "../../modules/resource-group"

  name     = local.resource_group_name
  location = var.location
  tags     = local.tags
}
```

This modular structure promotes reuse, consistency and simplified maintenance.

Continuous Integration

Application builds are orchestrated through GitHub Actions.

Each application is built independently using a matrix strategy before being containerised.

```
strategy:
  matrix:
    app:
      - auth-service
      - main-app
      - cleanup-job
```

This enables multiple applications to be built in parallel while sharing a single workflow definition.

Each build produces a Docker image before exporting it as a pipeline artefact.

```
- name: Build Docker Image
  uses: docker/build-push-action@v6
```

Only the `main` branch is permitted to publish images to Azure Container Registry.

if: github.ref == 'refs/heads/main'

This ensures feature branches cannot publish production images.

Authentication

Authentication between GitHub Actions and Azure is performed using OpenID Connect (OIDC).

Rather than storing Azure credentials inside GitHub, temporary federated identity tokens are exchanged with Microsoft Entra ID.

```
- uses: azure/login@v2
  with:
    client-id: ${{ vars.AZURE_CLIENT_ID }}
    tenant-id: ${{ vars.AZURE_TENANT_ID }}
    subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
```

This removes the need for long-lived service principal secrets while following Microsoft's recommended authentication model.

Container Registry

Container images are published into Azure Container Registry (ACR).

Each image receives:

- Immutable Git SHA tag
- Rolling `latest` tag

This enables repeatable deployments while supporting rollbacks to previous image versions if required.

GitOps Deployment

Rather than deploying directly from GitHub Actions, deployments are managed using ArgoCD.

GitHub Actions updates Kubernetes manifests with the newly published image version.

The Python automation script:

update_manifests.py

automates this process.

ArgoCD continuously monitors the repository and synchronises changes into the existing AKS cluster.

This establishes Git as the single source of truth while enabling:

- Automatic reconciliation
- Drift detection
- Rollback capability
- Self-healing deployments

Kubernetes

The platform deploys three Kubernetes workloads.

Application	Kubernetes Resource
Auth Service	Deployment
Main Application	Deployment
Cleanup Job	CronJob

Each Deployment includes:

- ReplicaSets
- Services
- Startup Probes
- Readiness Probes

- Liveness Probes

Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
```

These probes improve application resilience by ensuring only healthy pods receive traffic while automatically restarting failed containers.

Operational Automation

Operational tasks are automated using Python.

The repository currently includes:

```
automation/
├── update_manifests.py
├── health_check.py
└── acr_cleanup.py
```

These scripts automate repetitive operational activities including:

- Updating Kubernetes image versions
- Performing application health validation
- Removing unused container images

Automation reduces manual intervention while improving operational consistency.

Observability

Operational visibility is achieved through an integrated monitoring stack.

Prometheus continuously collects metrics from Kubernetes workloads.

Grafana presents dashboards for infrastructure and application performance.

Loki aggregates application logs from all services.

Alertmanager evaluates alert rules and forwards operational notifications to Slack.

This combination provides engineers with a centralised view of platform health while improving incident response and troubleshooting.

Security

Security is incorporated throughout the delivery pipeline.

Implemented controls include:

- Terraform-managed infrastructure
- OIDC authentication
- Azure RBAC
- Private Azure Container Registry
- GitHub Actions workflow permissions
- Kubernetes health probes
- Trivy container security scanning
- Renovate dependency management

These controls reduce operational risk while aligning with modern cloud security practices.

Technology Integration

The platform combines Azure managed services with open-source technologies to provide a secure, automated and maintainable deployment pipeline.

Technology	Purpose
Microsoft Azure	Cloud Platform
Terraform	Infrastructure as Code
GitHub Actions	Continuous Integration
Docker	Containerisation
Azure Container Registry	Container Image Storage
Kubernetes (AKS)	Container Orchestration *(Assumed Existing)*

ArgoCD	GitOps Deployment *(Assumed Existing)*
Prometheus	Metrics Collection *(Assumed Existing)*
Grafana	Dashboards *(Assumed Existing)*
Loki	Centralised Logging *(Assumed Existing)*
Alertmanager	Alert Routing *(Assumed Existing)*
Trivy	Container Security Scanning
Renovate	Dependency Management
Python Automation	Operational Automation

Conclusion

The proposed architecture demonstrates a modern cloud-native software delivery platform built around automation, Infrastructure as Code and GitOps principles. Azure provides the underlying cloud services while Terraform provisions infrastructure, GitHub Actions performs continuous integration, Docker packages the applications and Azure Container Registry stores versioned container images. Kubernetes, managed through ArgoCD, delivers highly available workloads, while Prometheus, Grafana, Loki and Alertmanager provide operational visibility through monitoring, logging and alerting.

By combining managed Azure services with open-source technologies, the platform remains secure, scalable and maintainable while minimising operational overhead. The clear separation between infrastructure provisioning, application delivery and runtime operations ensures that each component can evolve independently, making the solution suitable for enterprise environments and future growth.